

CASE: Reliability Engineering for Information Systems

Elliot J. Chikofsky and Burt L. Rubenstein, Index Technology

You can use CASE environments in new ways to support the entire software life cycle, including reliability engineering, maintenance, documentation, and auditing.

Developers of information systems have historically been the last to apply computer-based tools to improve the quality, reliability, and productivity of their own work — a phenomenon known as the “shoemaker’s children” syndrome, from the story of the shoemaker whose children never had new shoes because all new shoes were made for others. This tradition is changing as organizations involved in both commercial data processing and embedded real-time applications review their heavy investment in software and information systems. Having recently identified information systems as a capital resource, they now realize their need for surer design and reliability. By introducing new CASE technology, they can achieve greater system development productivity while providing a new approach to engineering information systems reliably.

As early as 1973, Barry Boehm projected that software costs would, by 1985, reach or exceed 90 percent of the

total cost of data processing (the combined hardware and software costs),^{1,2} as Figure 1 shows. It was clear that hardware costs were rapidly falling and that personnel costs in the labor-intensive software development arena were rising. The projection has turned out to be all too true.

Today, the rapid pace at which hardware innovations are announced, particularly in the area of microprocessor technology, now well exceeds the capabilities of our software-development technology. Consider, for example, the Intel 80286 processor, which can support multitasking operations. At the time of its introduction, the existing general-purpose operating systems for IBM PCs (such as MS-DOS and Xenix) had no way to use its capabilities. And then, before the first operating system (OS/2) was available to take full advantage of the 286, the 80386 processor made its debut with even greater capabilities. An entire generation of processor hardware technology has arrived and been superseded without any

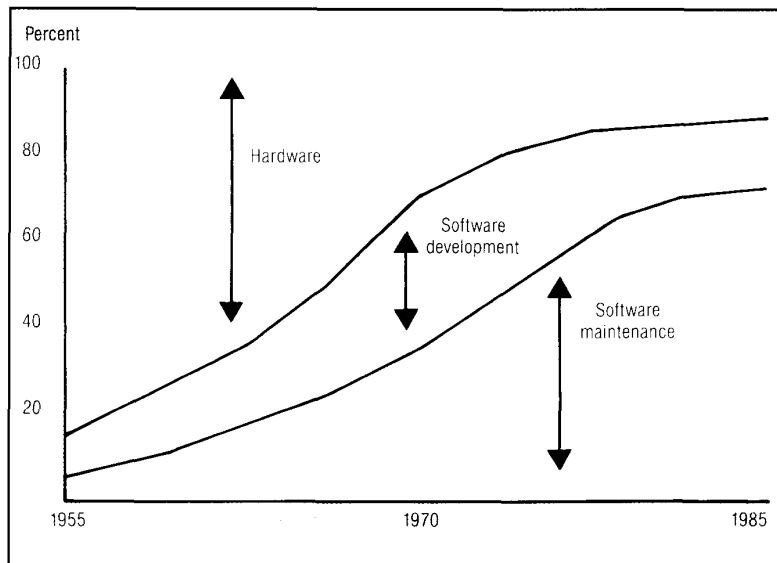


Figure 1. Evolution of system-development costs (y-axis is percent of total cost).

software to support it reaching the marketplace!

Further, the ability to develop software to meet new needs is rapidly losing ground to software maintenance. The US Air Force's F-16 jet fighter, for example, has an \$85-million budget for software development. On top of this, the Air Force is expecting to spend about \$250 million in software maintenance — almost three times the development cost.³

Organizations are now recognizing how the software crisis affects them. Randall Jensen of Hughes Aircraft, for example, has identified six key software problems resulting from systems-development approaches as commonly applied:⁴

- products exceeding cost estimates,
- late delivery,
- inadequate performance,
- impossible maintenance,
- prohibitive modification costs, and
- unreliability.

More than ever before, organizations are looking for workable solutions to increase the productivity of their systems-development staff, to improve the cost-effectiveness of the development process, and to ensure the reliability and quality of systems produced.

Development approaches

Since the mid-1970s, the approach to developing information systems has evolved with the growth of methods to aid

both project management and specification. Organizations have turned to more formal methods of system development and, most recently, to automated tools to support these methods.

Classical methods. The development of information systems and software has long been viewed as an art rather than a science. The programmer or analyst would wade through a loose collection of user wants and needs. Some time later, through some arcane and mystical process, a system would appear that, even though it did not meet all the user's needs, was usually considered better than what existed before. This classical approach was characterized by informal guidelines, lack of standardization, and minimal documentation (usually produced as an afterthought).

Some order was introduced in the early 1970s in the form of project-management methods based on models of the system-development life cycle. Since then, many packaged life-cycle methods — incorporating directions, forms, and acceptance standards — have been marketed commercially. Military standards for specification and documentation practices in use today are examples of content-based life-cycle methods. These methods try to manage the development process by requiring that documents be produced at specific checkpoints, according to prescribed forms or document-content guidelines.

The effectiveness of life-cycle methods

has been limited by two key problems: lack of formality, resulting in inconsistency, and lack of maintainability. Most life-cycle methods stress the capture of documentation during the development process but do not adequately provide for the continued usefulness of the documents produced. The result is often notebooks full of completed forms or a set of Victorian novel-like narrative specifications, both of which are destined to sit on bookshelves gathering dust. The specifications produced are often incomplete, inconsistent, and incorrect, and are seldom updated with changes to the system.

Formal methods. The need for greater productivity in the system-development life cycle led to the introduction of more formal methods in the late 1970s and early 1980s. Most methods were aimed at specific stages of the life cycle and were based on different user viewpoints about the target information systems.

Structured analysis, introduced popularly by Tom DeMarco,⁵ applied some of the decomposition techniques that improve programming to the front-end requirements and analysis phases of the systems life cycle. Structured analysis grew out of structured design, which itself had grown out of structured programming. Through the use of structured dataflow diagrams and descriptions of data, the analyst could build up a systematic description of the information system's logical (functional) and physical (implementation) aspects. Management procedures provided manual validation of the dataflow diagrams via walkthroughs, and diagrams and data descriptions became deliverable parts of system documentation.

Many variations of formal methods now exist. Well-known versions of structured analysis include the Yourdon/DeMarco and Gane/Sarson methods. Recent adaptations of structured analysis to specific needs of real-time systems development include the Ward/Mellor and Hatley methods. Data-oriented methods, applying similar principles to both data-centered design and data administration, include the Warnier/Orr, Holland, Bachman, and Martin's information-

engineering approaches. The realm of structured design has the Yourdon/Constantine and Jackson methods.

Much fanfare accompanied the introduction of formal methods, but their manual nature has limited their general use and acceptance. Dataflow diagrams developed manually are difficult to modify and are seldom maintained with the system they represent. The effort and cost to develop complete and useful descriptions of an evolving system are hard to justify when the information is clearly not a lasting part of maintenance.

What CASE offers

With the spread of desktop computers, the mid-1980s have seen the introduction of automated environments and tools that make it practical and economical to use formal system-development methods. This technology, known as CASE, lets systems analysts document and model an information system from its initial user requirements through design and implementation and lets them apply tests for consistency, completeness, and conformance to standards.

These techniques for software engineering apply equally well to modeling hardware and human interfaces, so the acronym also refers to computer-aided systems engineering. Integrated in this field are models, tools, and techniques that open new areas for applying reliability engineering to information systems.

Much like the children of the shoemaker who were always the last to get new shoes, computer professionals have traditionally been the last ones to apply the benefits of computer technology to the improvement of their own work. Now, however, with personal workstations becoming a common resource, CASE environments let computer professionals develop and validate system designs and specifications, automating and enhancing the manual methods of the 1970s and 1980s.

Features. A CASE environment provides the analyst or systems developer with facilities for drawing a system's architectural diagrams, describing and defining functional and data objects, identifying

relationships between system components, and providing annotations to aid project management.

The user's various work products are stored in an integrated, nonredundant form in a central repository or dictionary on the workstation or on a central server or host system. The system definition as a whole can be checked for consistency and completeness. Analysis can be performed on the information collected or defined to date, thus supporting incremental development and the detection of inconsistencies and errors early in the life cycle. Documentation required by organiza-

Computer professionals are usually the last to apply the benefits of computer technology to their own work. CASE is one of those benefits.

tional or deliverable standards, such as DoD-Std 2167, can be generated from the system description in the dictionary. Also, generators for database schemas and program code are being incorporated in, or interfaced to, CASE environments to provide a step toward automated system generation. Figure 2 shows key elements in a CASE system.

Tools. A comprehensive CASE development environment for the front end of the life cycle integrates several component tools and facilities:

- The system developer can choose from at least seven basic diagram types, including dataflow diagrams (Yourdon/DeMarco or Gane/Sarson notations), structure charts (Yourdon/Constantine or Jackson), entity-relationship diagrams, logical data models, and presentation graphs. Additional diagram types are available for real-time system development, including transformation graphs, state-transition diagrams, and decision matrices. The user can directly create dia-

grams with the graphics-editing facilities on the workstation and then produce those diagrams for system documentation. Figures 2 and 3 demonstrate these graphics capabilities.

- Information about the user's target system, entered incrementally via diagrams and form screens, is integrated in a central dictionary.

- End-user screens and reports can be developed for the system under design with screen-paint facilities tied to the central dictionary. These form functional prototypes on the workstation that can be demonstrated to end users for evaluation and validation.

- Analysis facilities provide checks for consistency and completeness. Entity-list query capabilities combined with a general report writer allow user-definable analysis that can be tailored to meet project or organizational needs. Extended analysis utilities provide advanced conclusions about the integrity of a proposed design, including validation of data normalization and traceability of data elements through activities in the target-system description.

- Deliverable documentation can be organized graphically and can incorporate diagrams and text from the central dictionary. This facility lets the user generate documentation that is internally consistent and true to the system design. The environment provides links to desktop-publishing, page-layout, and Postscript output facilities.

- An open architecture for access to the central dictionary allows the integration of the CASE environment with other tools of the project or the organization. Import/export facilities let you bridge to other programs, transfer information among multiple analysts, and access other information resources, such as the corporate data dictionary (Figure 3 shows data-dictionary integration). In addition, a programmer interface lets you access the system description in the central dictionary from application programs to allow transfer of data, broader analysis, and specialized documentation.

- Established bridges and integrated packages connect the front-end environment to many tools used in later stages of the life cycle. Links to code-generation

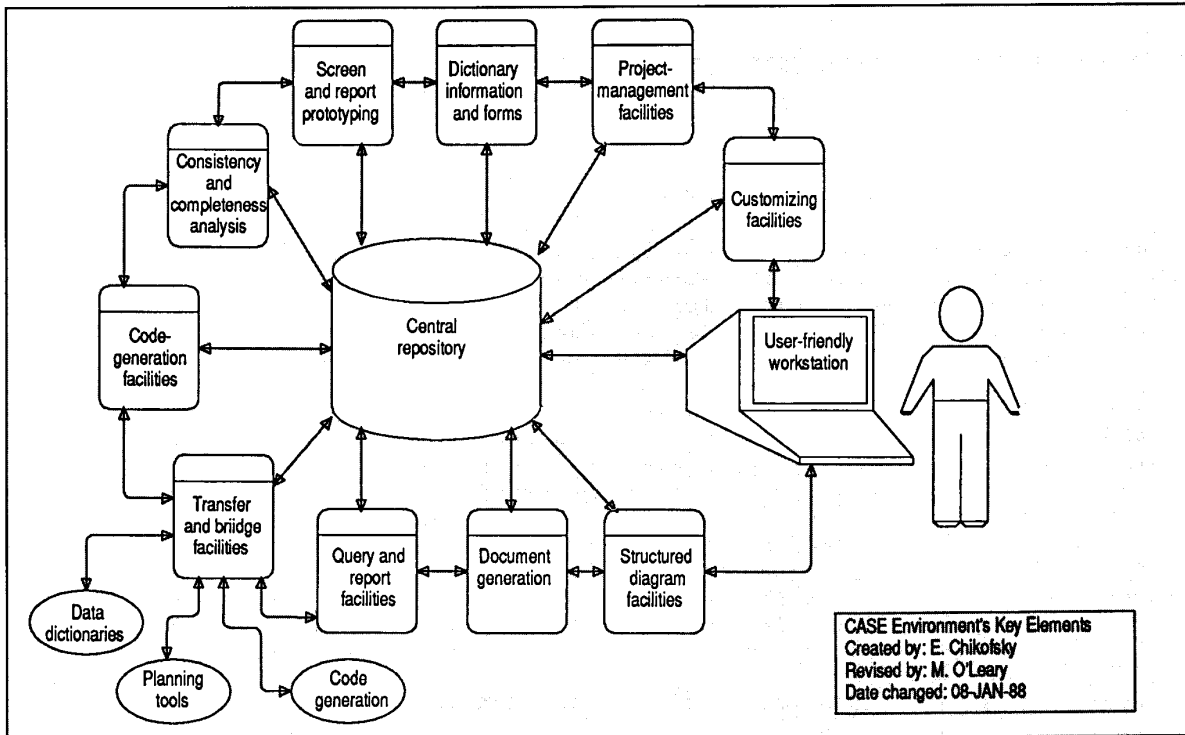


Figure 2. Key elements in a CASE system. This figure was generated from Index Technology's Excelerator CASE system and represents current CASE graphics capabilities.

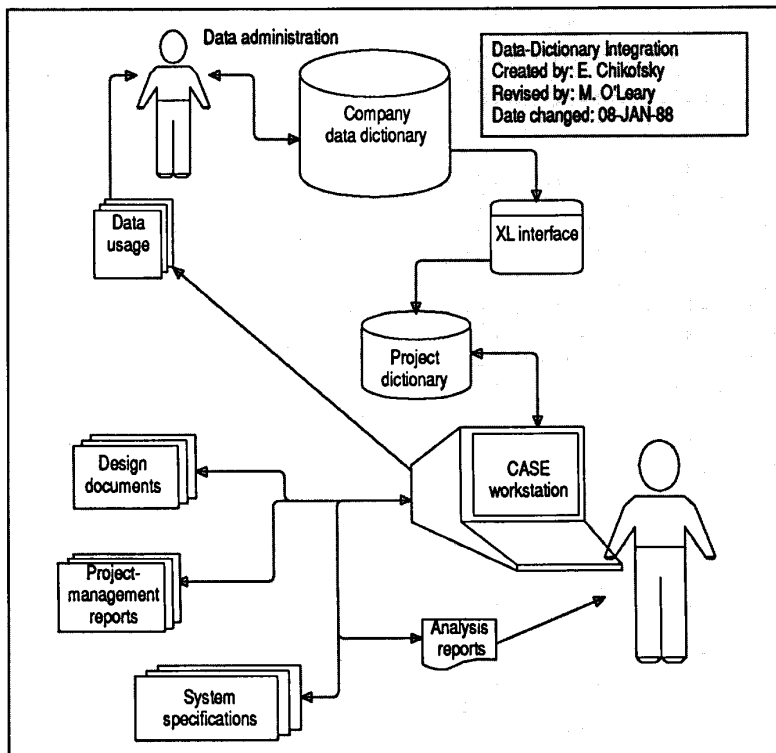


Figure 3. Data-dictionary integration. This figure was generated from Index Technology's Excelerator CASE system and represents current CASE graphics capabilities.

tools and fourth-generation languages, as well as to other types of development tools, give users a wide variety of options.

Adaptability. A key feature of a well-developed CASE environment is adaptability. To be successful in a wide range of organizations and projects, the environment must be able to operate on many hardware configurations in a consistent and user-friendly manner. To serve today's development organizations, the same base environment should be available on IBM PC-class and PS/2-class machines, on engineering workstations from Sun, Apollo, and Digital Equipment Corp., and on a variety of local-area networks.

Another important aspect of adaptability is how well the CASE environment can conform to an organization and project, rather than how much it makes the project conform to the idiosyncrasies of the tool. The CASE environment needs to be customizable so the user can adapt it to closely fit the organization's development method, project-management standards, and information requirements. This versatility lets a single CASE environment support the organization even as projects, standards, and contractual obligations change.

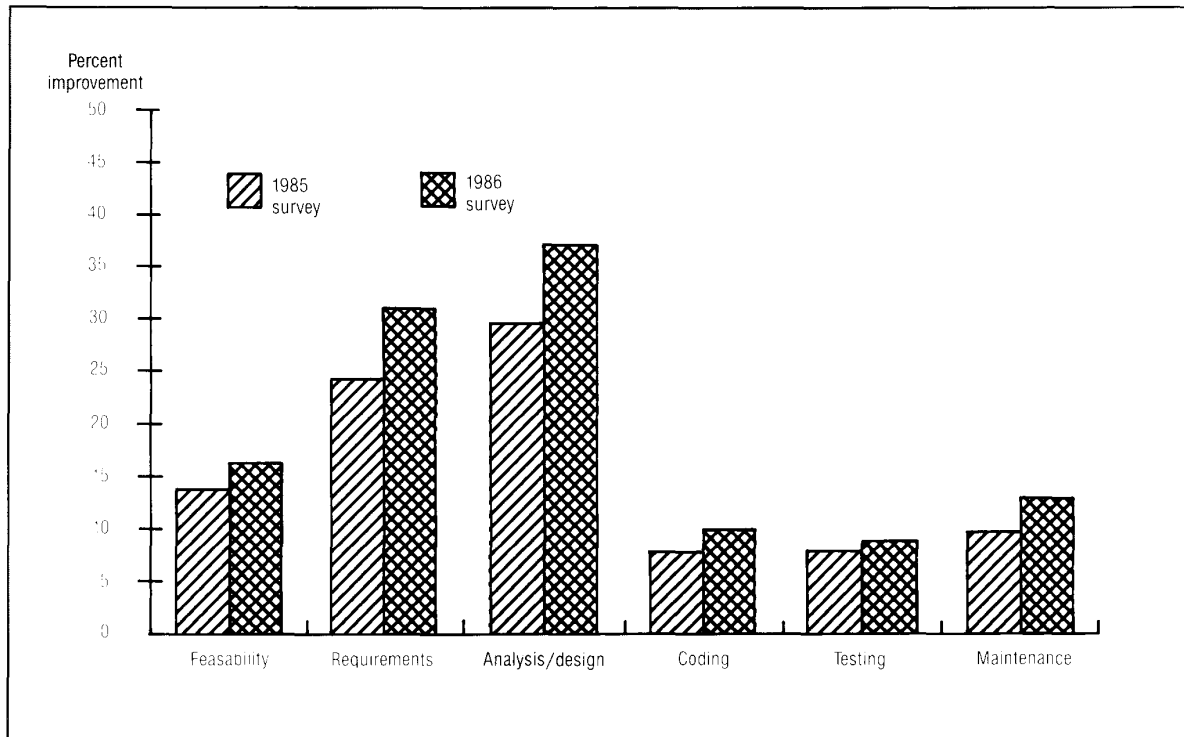


Figure 4. Productivity improvement in a system's life-cycle phases when using a CASE tool (sources are two surveys of users of Index Technology's Excelsator product).

Benefits. Organizations using CASE environments have experienced various degrees of improvement in productivity. Users of Index Technology's Excelsator environment, for example, have reported in surveys an average of 30- to 40-percent improvement in the life cycle's analysis and design phases (see Figure 4). But many of the user organizations also report that their key gain is in the quality of the systems being developed, principally because they can detect errors and inconsistencies and refine specifications easily to reflect their own customer's needs.

Reliability engineering

Besides serving as productivity aids to support the capture of system-design information, CASE environments provide new opportunities to use analysis techniques to assess the reliability of information systems before they are implemented. They also help audit a completed system against its design and maintain the system description as accurate documentation.

Analysis. A CASE environment stores the systems description in an integrated form in a central repository or dictionary. It provides native facilities for inspecting and analyzing aspects of the system description, particularly in support of standard development methods such as

structured analysis.

With the system description in the dictionary, other forms of inspection can be made using database query techniques. For example, if the correctness of timing factors is important to a system's success, an analyst can target those portions of the description that affect timing. The dictionary can provide information not only about the specific timing attributes but also about all other components that affect these attributes.

An analyst can also discover secondary effects, where the impact of an attribute was not explicitly defined but was implicit in the design of other parts of the system. The CASE environment thus forms a vehicle for reliability assessment as well as for system description.

The CASE environment can help standardize quality assurance and test processes for systems under development. By using the available system description for test planning, aided by automated CASE facilities, an organization can develop more rigorous coverage for test cases and can better determine if the features and components being tested match important paths in the system.

Maintenance. When a system description has been completed, the CASE environment can continue to serve as an active

repository and reference encyclopedia for the system's builders and maintainers. The ease of making changes to the system description lets the documentation be maintained with the system. When maintenance staff can use an up-to-date system description as a source of information on the system architecture, they are encouraged to update it for their own benefit as they make system modifications. The result is more shoemaker's children with new and valuable shoes.

Audit. The same principle applies to auditing. With an up-to-date system description supported by a CASE environment, the dictionary can serve as a key resource and record-keeping vehicle. Various system components and their operation can be compared to their intended function in the system description, and notes can be made in an auditor's copy of the dictionary. The completed system can be more effectively evaluated against its design, and automated assistance can be provided to trace requirements through design and into the finished system. Thus, the CASE environment also becomes an auditor's tool. This advantage can be further enhanced by customizing the CASE environment to handle the specific attributes required of the auditing or reliability evaluation task at hand.

The RATIONAL Environment...

for Software Engineers

The Company

Rational was founded in 1980 to produce products which improve the productivity of software engineering teams building large and complex systems. The R1000 Development System is now in use by leading aerospace, commercial and defense organizations worldwide.

The Product

Rational's R1000 provides a seamlessly integrated environment for the design, implementation, documentation, and maintenance of large software systems written in Ada. It provides comprehensive facilities across the software lifecycle, including an Ada-compatible program design language, automatic generation of design documentation, interactive semantic analysis and design-rule checking, incremental compilation, configuration management and version control, and source-language debugging. Optimizing code generators and cross-debuggers provide support for several popular application architectures.

The People

Rational is a team of highly motivated, creative individuals dedicated to the production of quality products that deliver value to customers. We are looking for Software Engineers with strong experience in operating systems, design tools code generation or debuggers to join this team.

If you'd like to be a part of our future, please write us at 1501 Salado Dr., Mountain View, CA 94043. Attn: Human Resources. An equal opportunity employer.

RATIONAL

Emerging technology in CASE provides new and potentially invaluable support to aspects of systems work throughout the systems life cycle. There are broad opportunities for automating reliability-engineering tasks for information systems. As this technology matures, we can look forward to the incorporation of meaningful expert-system and artificial-intelligence capabilities to help the systems analyst, libraries of reusable system designs, and automated prototyping of systems to help determine reliability.

Existing developments in CASE put these possibilities within our grasp and offer a bright future for the entire area of information-systems reliability. □

References

1. B. Boehm, "Software and Its Impact: A Quantitative Assessment," in *Writings of the Revolution*, Yourdon Press, New York, 1982, pp. 267-289.
2. B. Boehm, *Software Engineering Economics*, Prentice-Hall, Englewood Cliffs, N.J., 1981.
3. W. Suydam, "CASE Makes Strides toward Automated Software Development," *Computer Design*, Jan. 1, 1987, pp. 49-70.
4. R. Jensen, "Predicting and Controlling Software Development Costs: Metrics for Managers," *Proc. Second Ann. Excelsior Users' Conf.*, XL Group, Cambridge, Mass., 1986.
5. T. DeMarco, *Structured Analysis and System Specification*, Yourdon Press, New York, 1978.



Elliot J. Chikofsky is director of research and technology at Index Technology and teaches graduate courses in software engineering and database management at Northeastern University. Before joining Index Technology, he worked on the University of Michigan's ISDOS project.

Chikofsky received a BS in computer and communication science and an MS in industrial and operations engineering, both from the University of Michigan.



Burt L. Rubenstein is a vice president of research and technology at Index Technology. He was the original architect of the Excelsior CASE system. He has also worked at Index Systems and Digital Equipment Corp.

Rubenstein received a BS in management and computer science from the Massachusetts Institute of Technology, where he also did systems research in the Laboratory for Computer Science.